

DOCUMENTO TÉCNICO DE SEGURANÇA

Levantamento de Segurança do Backend

Plataforma de Achados e Perdidos para Grandes Eventos — API (Spring Boot)

Versão	1.0
Data	17 de julho de 2026
Escopo	Backend / API REST
Referência	OWASP Top 10 (2021)
Classificação	Confidencial

1. Sumário Executivo

Este documento consolida a análise de segurança da camada de **backend** da plataforma, compara a implementação real com o que a documentação funcional descreve, detalha o tratamento do **OWASP Top 10 (2021)** presente no código e responde aos questionamentos técnicos levantados pelo cliente sobre arquitetura, persistência/armazenamento e pipeline.

A avaliação geral é positiva: a base de segurança está **acima da média para um MVP**, com JWT robusto, proteção a força-bruta, autorização granular por permissão, IDs de recurso assinados (anti-IDOR) e senhas com BCrypt. Durante a revisão foi identificada e **já corrigida** uma vulnerabilidade de leitura arbitrária de arquivo no módulo de anexos; os demais pontos são recomendações de *hardening* e de alinhamento entre documentação e implementação.

Destaques já implementados: autenticação JWT com *fail-fast* de segredo fraco • rotação e revogação de *refresh token* • rate limit de login por IP e por conta • autorização por permissão (`modulo.acao`) • IDs assinados por HMAC • SCA de dependências (Trivy) no pipeline • BCrypt • CORS configurável • auditoria de acesso.

2. Correção Aplicada Nesta Revisão

A revisão identificou e corrigiu, no módulo de arquivos (`ArquivoService` / `ArquivoController`), os seguintes pontos:

Item	Sev.	Descrição e correção
Path Traversal / leitura de arquivo arbitrário	Crítico	O caminho físico do arquivo era resolvido sem confinamento na pasta base, permitindo que um metadado com <code>nmPath</code> absoluto (ex.: <code>/etc/passwd</code>) ou com <code>../</code> lesse arquivos fora do diretório de anexos. Correção: método central <code>resolverDentroDaBase()</code> que normaliza e exige que o caminho permaneça dentro de <code>app.arquivos.dir</code> , recusando caminhos absolutos e <i>traversal</i> . Aplicado no upload e no download.
XSS armazenado via download	Alto	O download servia o conteúdo com <code>Content-Disposition: inline</code> e o <i>content-type</i> gravado, permitindo renderizar HTML/SVG malicioso no domínio da API. Correção: <code>attachment</code> + header <code>X-Content-Type-Options: nosniff</code> e sanitização do nome do arquivo no cabeçalho.
Upload sem validação de tipo	Médio	Qualquer tipo de arquivo era aceito. Correção: <i>allowlist</i> de MIME (JPEG, PNG, WEBP, GIF, HEIC e PDF).

Correção validada com `mvn compile` (BUILD SUCCESS). Arquivos alterados: `service/ArquivoService.java` , `controller/ArquivoController.java` .

3. OWASP Top 10 (2021) — Como É Tratado no Backend

A tabela abaixo mapeia cada categoria do OWASP Top 10 ao controle correspondente **já presente no código**, com a evidência (arquivo). A legenda de status: **Coberto** **Parcial / recomendação** **Corrigido nesta revisão** .

Categoria	Status	Controle implementado & evidência
A01 — Broken Access Control	Coberto Fix	Autorização por permissão granular <code>@PreAuthorize("@authz.pode('modulo.acao'))</code> com <code>@EnableMethodSecurity</code> ; regras por rota em <code>SecurityConfig</code> ; IDs de recurso assinados por HMAC (impedem IDOR/enenumeração); path traversal de arquivos confinado. <code>SecurityConfig.java</code> , <code>AuthorizationService.java</code> , <code>SignedResourceIdCodec.java</code> , <code>ArquivoService.java</code>
A02 — Cryptographic Failures	Coberto	Senhas com BCrypt ; JWT assinado HS256 com segredo forte obrigatório (<i>fail-fast</i> se ausente/curto/exemplo); IDs assinados com HMAC-SHA256 ; segredos sem valor default (não sobem sem variável de ambiente). <code>JwtUtil.java</code> , <code>SignedResourceIdCodec.java</code> , <code>application.properties</code>
A03 — Injection	Coberto Fix	Acesso a dados via Spring Data JPA parametrizado (sem concatenação de SQL); Bean Validation (<code>@Valid</code>) nas DTOs; <code>fail-on-unknown-properties</code> rejeita campos extras; XSS de download mitigado (attachment + nosniff). <code>repository/*</code> , <code>GlobalExceptionHandler.java</code> , <code>ArquivoController.java</code>
A04 — Insecure Design	Coberto	Tokens tipados (access / refresh) com rotação e revogação de refresh via <code>jti</code> persistido; rate limit aplicado antes da verificação de credenciais; contato de confiança no fluxo de claim. <code>AuthService.java</code> , <code>LoginRateLimiter.java</code>
A05 — Security Misconfiguration	Coberto Parcial	Recusa de segredos de exemplo; <code>ddl-auto=validate</code> ; <code>open-in-view=false</code> ; CSRF desabilitado corretamente (API stateless com Bearer); CORS configurável; Swagger desligável em produção (<code>SPRINGDOC_ENABLED</code>); allowlist de MIME; headers de segurança (HSTS, nosniff, frame-options DENY, Referrer-Policy) adicionados. Recomenda-se ainda <code>useSSL=true</code> no MySQL. <code>SecurityConfig.java</code> , <code>JwtUtil.java</code> , <code>application.properties</code> , <code>CorsConfig.java</code>
A06 — Vulnerable & Outdated Components	Coberto Parcial	Stack recente (Spring Boot 4.0.2, JDK 25); pipeline de segurança com SCA (Trivy) , SAST (Semgrep) e secret scanning (gitleaks) . Jobs visíveis a cada execução (<code>allow_failure</code> na fase de triagem; viram gate bloqueante ao remover o flag). <code>pom.xml</code> , <code>.gitlab-ci.yml</code>
A07 — Identification & Authentication Failures	Coberto	Proteção a força-bruta (Bucket4j) por IP e por conta ; mensagem genérica anti-enenumeração de usuário; JWT stateless; BCrypt; refresh rotativo e revogável. <code>LoginRateLimiter.java</code> , <code>AuthController.java</code> , <code>AuthService.java</code>
A08 — Software & Data Integrity Failures	Coberto Fix	Integridade/anti-adulteração dos IDs via HMAC com comparação em tempo constante (<code>MessageDigest.isEqual</code>); refresh tokens persistidos e revogáveis; validação de MIME no upload; SCA de dependências (cadeia de suprimentos). <code>SignedResourceIdCodec.java</code> , <code>ArquivoService.java</code> , <code>.gitlab-ci.yml</code>
A09 — Security Logging & Monitoring Failures	Coberto Parcial	Registro de acesso no login (IP, dispositivo, User-Agent); módulo de auditoria de ações. Recomenda-se auditar também o acesso a dados sensíveis (exigência LGPD/documentação). <code>AuthService.java</code> , <code>LoginLogService</code> , <code>entity/Auditoria.java</code>
A10 — Server-Side Request Forgery (SSRF)	Baixa exposição	Não há requisições de saída construídas a partir de entrada do usuário — superfície mínima. Ao integrar e-mail transacional/S3, os <i>endpoints</i> devem ser fixados por configuração (nunca derivados de entrada do cliente). —

4. Respostas aos Questionamentos

4.1 “A documentação fala em microsserviços, mas parece monólito.”

Resposta: Correto. Tecnicamente, o backend é um **monólito modular** — uma única aplicação Spring Boot, bem separada em camadas (`controller` → `service` → `repository` → `entity`), e **não** um conjunto de microsserviços independentes. A separação “clara entre camadas” existe; o termo “microsserviços” na documentação é impreciso e **deve ser ajustado**.

Isso **não é uma limitação** para o estágio atual, e não impede escala horizontal: a API é **stateless** (autenticação por JWT, sem sessão em memória), então pode rodar em N réplicas atrás de um *load balancer*. As fronteiras de módulo já existentes facilitam uma eventual extração para microsserviços no futuro, se o volume justificar.

Ressalva de escala: dois pontos hoje são “stateful” e precisam de atenção antes de escalar para múltiplas réplicas: (1) o **rate limit de login é em memória por instância** — para um limite global, migrar para um store distribuído (ex.: Redis); (2) os **anexos são gravados em disco local** — ver 4.2.

4.2 “Os dados que não vêm do repositório não podem ir para o AWS S3? Isso adiciona muita complexidade de vocês?”

Resposta: Sim, podem — e essa é, inclusive, a abordagem recomendada. Hoje os **dados relacionais** (itens, claims, usuários, auditoria) ficam no **MySQL** e continuam lá; o que grava em **disco local** são apenas os **binários** (fotos de itens e comprovantes de posse), no diretório `app.arquivos.dir`. É exatamente isso que obriga a tratar volume persistente (PVC) e prende as réplicas ao mesmo disco.

Migrar esses binários para **AWS S3** (ou compatível, como MinIO) **não adiciona complexidade relevante do nosso lado**, porque todo o I/O de arquivos já está isolado em um único componente (`ArquivoService` , nos métodos de upload e leitura). A mudança se resume a:

- Introduzir uma interface `StorageService` com uma implementação para S3 (AWS SDK v2);
- Trocar a gravação/leitura em disco por `putObject` / `getObject` — o restante do fluxo (autorização, metadados no banco, IDs assinados) permanece idêntico;
- Opcionalmente, gerar **URLs pré-assinadas** para download, tirando o tráfego de binários da API e mantendo o mesmo modelo de permissão.

Ganhos: réplicas 100% *stateless* (sem PVC), durabilidade e *backup* gerenciados pelo S3, e simplificação da infraestrutura de contêiner. Estimativa de esforço: **baixa** — é um adaptador localizado, não uma reescrita. Observação: o módulo de *Wallpaper* já é “zero servidor” (processado no navegador), então o único módulo que realmente precisa de armazenamento externo é o de **anexos**.

4.3 “Ainda não ficou claro como tratam o OWASP Top 10.”

Resposta: O tratamento está detalhado na **Seção 3** deste documento, categoria por categoria (A01 a A10), cada uma com o controle correspondente e o **arquivo de evidência** no código — não é um selo de marketing, e sim controles verificáveis. Em resumo: acesso e autenticação são os pontos mais fortes (JWT, rate limit, autorização granular, IDs assinados); os pontos com recomendação aberta são *hardening* de configuração (headers HTTP, TLS no banco), scan bloqueante no pipeline e auditoria de acesso a dados sensíveis para LGPD.

4.4 “Prevenção com avaliação contínua de risco: há etapa de validação de código no pipeline? Se não, como é feito?”

Resposta: O pipeline (`.gitlab-ci.yml`) tem 4 estágios — `build` → `test` → `security` → `deploy` — e o estágio de segurança foi **ampliado nesta revisão**. Situação atual:

Verificação	Situação	Observação
Testes automatizados (gate)	Presente + reforçado	O estágio <code>test</code> agora sobe um MySQL efêmero no CI, então os testes de integração realmente executam (antes seriam pulados por falta de banco). Falha de teste bloqueia o deploy.
SCA — dependências (A06)	Presente	Trivy (HIGH/CRITICAL) agora com <code>--exit-code 1</code> — o job fica visível/vermelho quando há CVE.
SAST — análise estática (A05/A08)	Adicionado	Semgrep com regras públicas de Java, segredos e OWASP Top 10.
Secret scanning	Adicionado	gitleaks — detecção de chaves/tokens/senhas commitados.
DAST	Opcional	Ainda ausente — sugerido OWASP ZAP contra <i>staging</i> em fase posterior.

Estratégia de rollout: os três jobs de segurança (SCA, SAST, secret scanning) entram como `allow_failure: true` — ou seja, **rodam e ficam visíveis** a cada pipeline, mas **não bloqueiam** a entrega enquanto a equipe faz a primeira triagem de achados. Depois disso, cada um vira um **gate bloqueante** apenas removendo o `allow_failure`. Assim a “avaliação contínua de risco” passa a ser contínua de fato, sem travar o time no dia da virada.

Próximos passos do pipeline: (1) promover SCA/SAST/secret a bloqueantes após triagem; (2) adicionar DAST opcional em *staging*.

Sobre a infraestrutura do pipeline (o que o cliente precisa considerar): todas essas etapas rodam como **imagens Docker** no próprio GitLab CI — não exigem infraestrutura adicional além de um *runner* com Docker. A validação de código pode permanecer **inteiramente no nosso pipeline**; o cliente não precisa criar uma etapa própria. Caso exista um pipeline “guarda-chuva” do lado do cliente para promover à produção, ele pode **consumir os relatórios** (SAST/SCA em formato SARIF/JSON) como *gate* de qualidade.

5. Riscos Remanescentes & Roadmap de Hardening

Item	Prioridade	Ação recomendada
Rate limit distribuído	Alta*	Migrar Bucket4j para Redis quando houver múltiplas réplicas (limite global). *Alta apenas ao escalar.
Rate limit nos endpoints públicos do portal	Concluído	Limite por IP e ação (Bucket4j) aplicado em claims, crianças e registro. Próximo passo opcional: CAPTCHA no registro.
Headers de segurança HTTP	Concluído	HSTS, nosniff, frame-options DENY e Referrer-Policy adicionados no <code>SecurityConfig</code> .

Item	Prioridade	Ação recomendada
Confiança em <code>X-Forwarded-For</code>	Concluído	<code>server.forward-headers-strategy=NATIVE</code> : o Tomcat resolve o IP real só a partir de proxy interno confiável; removido o parsing manual e spoofável do header.
SAST / secret scanning / gate de teste no pipeline	Concluído	Ver 4.4 — Semgrep, gitleaks e MySQL efêmero no CI para o gate de teste ser real.
Armazenamento de anexos em S3	Média	Ver 4.2 — habilita réplicas stateless e remove dependência de PVC.
TLS na conexão com o MySQL	Média	Trocar <code>useSSL=false</code> por conexão cifrada em produção.
Promover scans de segurança a bloqueantes	Baixa	Após triagem inicial, remover <code>allow_failure</code> de SCA/SAST/secret. DAST opcional.
LGPD — retenção & dados sensíveis	Média	Confirmar/implementar rotina de retenção/anonimização, criptografia em repouso de campos sensíveis e auditoria de acesso a esses dados (a documentação promete; a verificar no código).

6. Conclusão

O backend entrega uma fundação de segurança sólida e coerente com boa parte do que a documentação promete: os controles centrais de autenticação, autorização e integridade estão implementados e são verificáveis no código. A vulnerabilidade mais séria encontrada (leitura arbitrária de arquivo) foi **corrigida nesta revisão**. Os itens remanescentes são melhorias de *hardening* e de maturidade operacional — principalmente **SAST no pipeline**, **armazenamento externo (S3)** para escalar sem PVC, **rate limit distribuído** e as pendências de **LGPD** — todos de esforço controlado e sem reescrita de arquitetura. Recomenda-se, ainda, **ajustar a documentação** para refletir que a solução é um monólito modular *stateless*, e não microsserviços.

Documento gerado a partir da análise do código-fonte do backend. Classificação: Confidencial.